Task 1: Radar Spec Sheet

Background

A radar sends out a radio wave, waits for it to bounce off something, and listens for the echo. Before we design anything we need to know what our radar should be able to do: how far it can see, and how close together two objects can be before it mixes them up. Your job is to work those numbers out. Every other team will design their part around your answers.

You will need to open MATLAB or Excel, crunch some numbers and write them up in a quick, rough report. Other teams will use your numbers, so make sure you get them right. Once you're done, write the values up in a Word doc (or whatever you like) and put it on the GitHub.

Here is what the terms mean:

- **Frequency** is how many times per second the radio wave goes up and down. Ours is 24 GHz, which is 24 billion times per second.
- **Wavelength** is the distance the wave travels during one of those up-and-down cycles. Higher frequency means a shorter wavelength.
- **Bandwidth** is how far the radar sweeps its frequency. Ours sweeps from 24.000 to 24.250 GHz, so the bandwidth is 250 MHz. More bandwidth lets the radar tell apart objects that are closer together.
- **dBm** is a way of writing power on a logarithmic scale, because radar powers range from tiny to huge. 0 dBm is 1 mW. Every 10 dBm more is ten times the power, so 10 dBm is 10 mW, 20 dBm is 100 mW, and -30 dBm is 0.001 mW.
- **dB** works the same way but for ratios. 10 dB means "10 times bigger", 20 dB means "100 times bigger".
- **Antenna gain** says how well the antenna focuses the signal in one direction, like a torch compared with a bare bulb. It is given in dBi.
- **Radar cross section** (σ) is how big a target looks to the radar, measured in square metres. A person is about $1\,\mathrm{m}^2$.
- **Noise** is the random electrical hiss present in every circuit. The echo has to be stronger than the noise or we cannot see it.
- **Noise figure** is how much extra noise our receiver chip adds on top of that.
- **Signal-to-noise ratio (SNR)** is how many times stronger than the noise the echo must be before we trust it is a real target.
- **Minimum detectable signal (MDS)** is the weakest echo we can detect. It sets the maximum range.

Values to use

- Frequency: $f = 24\,\mathrm{GHz}$
- Bandwidth: $BW = 250\,\mathrm{MHz}$
- Transmit power: $P_t = 8\,\mathrm{dBm}$
- Antenna gain (transmit and receive): $G = 10\,\mathrm{dBi}$

- Receiver noise figure: $NF = 10\,\text{dB}$
- Required signal-to-noise ratio: $SNR = 10\,\text{dB}$
- Radar cross section of the target: $\sigma = 1\,\text{m}^2$
- Speed of light: $c = 3 \times 10^8\,\text{m/s}$

Equations

Wavelength of the radar signal:

$$\lambda = \frac{c}{f} \tag{1}$$

Range resolution (the smallest gap between two targets that the radar can still see as two separate targets):

$$\Delta R = \frac{c}{2\,BW} \tag{2}$$

Converting between dBm, dB and normal units. You must convert to watts and plain ratios before using the radar equation:

$$P_{\text{mW}} = 10^{\,P_{\text{dBm}}/10} \qquad P_{\text{dBm}} = 10\log_{10}(P_{\text{mW}}) \qquad G_{\text{linear}} = 10^{\,G_{\text{dBi}}/10} \tag{3}$$

Minimum detectable signal. B is the bandwidth of the receiver in Hz, and $-174\,\text{dBm/Hz}$ is the noise present in every circuit at room temperature:

$$MDS_{\text{dBm}} = -174 + 10\log_{10}(B) + NF + SNR \tag{4}$$

Radar equation for maximum range, where $P_{\min}$ is the MDS converted to watts:

$$R_{\max} = \left[\frac{P_t\,G_t\,G_r\,\lambda^2\,\sigma}{P_{\min}\,(4\pi)^3}\right]^{1/4} \tag{5}$$

The same equation rearranged to give the power received from a target at range R:

$$P_r = \frac{P_t\,G_t\,G_r\,\lambda^2\,\sigma}{(4\pi)^3\,R^4} \tag{6}$$

What to do

1. Calculate the wavelength λ and the range resolution ΔR.
2. Convert P_t from dBm to watts and the antenna gain from dBi to a plain ratio.
3. Calculate the MDS for receiver bandwidths of $B = 1\,\text{MHz}$, $100\,\text{kHz}$, $10\,\text{kHz}$ and $1\,\text{kHz}$. Convert each one to watts.
4. Calculate $R_{\max}$ for each of those bandwidths.
5. Calculate the received power P_r (in watts and in dBm) for a person at $1\,\text{m}$, $10\,\text{m}$ and $30\,\text{m}$.
6. Plot $R_{\max}$ against receiver bandwidth, and P_r against range.
7. Write one or two sentences on what happens to the maximum range as the receiver bandwidth gets smaller, and why.

What to hand in

A one-page spec sheet on the GitHub with a table of λ, ΔR, the MDS and $R_{\max}$ for each bandwidth, and P_r at each range, plus your two plots and your spreadsheet or MATLAB script.

Task 2: Design the Frequency Ramp

Background

The ADF5901 chip contains a voltage controlled oscillator (VCO). This is a circuit that generates a wave whose frequency depends on the voltage you put on its control pin (called VTUNE). Higher voltage, higher frequency.

We want a frequency modulated continuous wave (FMCW). This is a wave that starts at a low frequency (24.000 GHz), climbs steadily to a high frequency (24.250 GHz), then jumps back and repeats. If you plot frequency against time it looks like a sawtooth. Each climb is called a ramp.

The chip that makes this happen is the ADF4159. It is a phase-locked loop (PLL). It measures the frequency coming out of the VCO, compares it with a very accurate reference clock, and nudges the VTUNE voltage up or down until the VCO is at exactly the frequency we asked for. To make a ramp, the ADF4159 keeps asking for a slightly higher frequency in lots of small steps, like climbing a staircase.

One thing to watch: the ADF5901 sends the ADF4159 a copy of its output at *half* the frequency. So the ADF4159 works from 12.000 GHz to 12.125 GHz, a sweep of 125 MHz. Do all of your sums with these halved numbers.

The settings you need to find

We control the ADF4159 by loading numbers into it. Your job is to work out what those numbers are.

- **INT** and **FRAC** set the frequency at the bottom of the ramp. The chip multiplies its reference frequency by a number N. INT is the whole-number part of N and FRAC sets the part after the decimal point.
- **DEV** and **DEV_OFFSET** set the height of each step on the staircase, i.e. how much the frequency jumps each time. DEV is a whole number up to 2^{15}, and DEV_OFFSET (a whole number from 0 to 9) multiplies it by a power of two so that bigger steps are possible.
- **CLK1** and **CLK2** set how long the chip waits on each step before taking the next one. Both are whole numbers from 1 to 4095, and they cannot both be 1.
- **Number of steps** is how many steps make up one full ramp.

Values to use

- Reference frequency: $f_{PFD} = 100\,\text{MHz}$
- Start frequency at the ADF4159: 12.000 GHz
- Sweep at the ADF4159: $BW_{PLL} = 125\,\text{MHz}$
- Number of steps: 1024
- Ramp time: $T_{ramp} = 5\,\text{ms}$

Equations

Output frequency of the PLL:

$$f_{out} = \left(\text{INT} + \frac{\text{FRAC}}{2^{25}}\right) f_{PFD} \tag{7}$$

So the multiplier you need is

$$N = \frac{f_{out}}{f_{PFD}}, \qquad \text{INT} = \text{whole part of } N, \qquad \text{FRAC} = (N - \text{INT}) \times 2^{25} \tag{8}$$

Frequency resolution (the smallest change in frequency the chip can make):

$$f_{RES} = \frac{f_{PFD}}{2^{25}} \tag{9}$$

Size of each frequency step:

$$f_{DEV} = \frac{BW_{PLL}}{\text{number of steps}} \tag{10}$$

DEV_OFFSET, rounded *up* to the next whole number (use 0 if the answer is negative):

$$\text{DEV_OFFSET} = \log_2\left(\frac{f_{DEV}}{f_{RES} \times 2^{15}}\right) \tag{11}$$

DEV, rounded to the nearest whole number:

$$\text{DEV} = \frac{f_{DEV}}{f_{RES} \times 2^{\text{DEV_OFFSET}}} \tag{12}$$

The step size you actually get after rounding:

$$f_{DEV,\,actual} = f_{RES} \times \text{DEV} \times 2^{\text{DEV_OFFSET}} \tag{13}$$

Time spent on each step:

$$t_{step} = \frac{T_{ramp}}{\text{number of steps}} \qquad \text{and} \qquad t_{step} = \frac{\text{CLK1} \times \text{CLK2}}{f_{PFD}} \tag{14}$$

What to do

1. Read the section "FMCW Radar Ramp Settings Worked Example" on page 27 of the ADF4159 datasheet. You are going to repeat that example with our numbers.
2. Calculate N at the bottom of the ramp (12.000 GHz) and at the top (12.125 GHz). Find INT and FRAC for the bottom of the ramp.
3. Calculate f_{RES} and f_{DEV}.
4. Calculate DEV_OFFSET and DEV, then the step size you actually get after rounding.
5. Multiply the actual step size by the number of steps. Does the ramp cover the full 125 MHz?
6. Calculate t_{step}, then the value of CLK1 × CLK2. It will not be a whole number, so pick whole numbers for CLK1 and CLK2 that get as close as possible, and work out the ramp time you actually get.
7. Remember the transmitted signal is at double the frequency. Write down the step size and the sweep as seen at the antenna.
8. Repeat steps 6 and 7 for ramp times of 1 ms and 10 ms, in case the sampling team asks for a different ramp time.
9. Extra: using the register maps on pages 15 to 24 of the datasheet, work out where each of your numbers goes in the 32-bit registers R0 to R6.

What to hand in

A table on the GitHub listing INT, FRAC, DEV, DEV_OFFSET, CLK1, CLK2 and the number of steps for each ramp time, with your working.

Task 3: Beat Frequency and Sampling

Background

Our radar does not send a single fixed frequency. It sweeps steadily from a low frequency to a high one, over and over. One sweep is called a ramp, and the ramp time T_{ramp} is how long the radar takes to get from its lowest frequency to its highest.

While the signal travels out to a target and back, the radar has carried on ramping. So by the time the echo returns, the radar is already transmitting a slightly higher frequency than the echo. The receiver chip (ADF5904) subtracts one from the other and outputs the difference. This difference is called the **beat frequency**. The further away the target, the longer the delay, and the higher the beat frequency. So if we measure the beat frequency, we know the range.

To measure it, a computer has to record the signal. It does this by **sampling**: taking a voltage reading many times per second. The part that does this is called an analogue-to-digital converter (ADC), and the number of readings per second is the **sampling rate**. If we sample too slowly, we miss the wiggles of the signal and get the wrong answer. The rule (called the Nyquist rule) is that we must sample at least twice as fast as the highest frequency we want to measure.

Once we have the samples, we use a piece of maths called the **FFT** (fast Fourier transform). You give it a list of samples and it tells you which frequencies are in the signal. A target shows up as a peak at its beat frequency.

There is a catch. A slow ramp gives low beat frequencies, which are easy to sample, but a slow ramp cannot keep up with fast-moving targets. A fast ramp handles fast targets but needs a much faster ADC. Your job is to work out the numbers and choose.

Values to use

- Bandwidth: $BW = 250\,\text{MHz}$
- Wavelength λ: get this from Task 1, or use $\lambda = c/f$ with $f = 24\,\text{GHz}$
- Speed of light: $c = 3 \times 10^8\,\text{m/s}$
- Ranges: $R = 1\,\text{m}$, $10\,\text{m}$ and $30\,\text{m}$
- Ramp times: $T_{ramp} = 1\,\text{ms}$, $5\,\text{ms}$ and $10\,\text{ms}$

Equations

Time for the signal to reach the target and come back:

$$\tau = \frac{2R}{c} \tag{15}$$

Slope of the ramp (how fast the frequency is changing, in Hz per second):

$$S = \frac{BW}{T_{ramp}} \tag{16}$$

Beat frequency:

$$f_b = S\,\tau = \frac{2\,R\,BW}{c\,T_{ramp}} \tag{17}$$

Rearranged to get the range back from a measured beat frequency:

$$R = \frac{c\,T_{ramp}\,f_b}{2\,BW} \tag{18}$$

Sampling rate needed (Nyquist rule):

$$f_s \geq 2\,f_{b,\max} \tag{19}$$

Number of samples taken during one ramp:

$$N = f_s\,T_{ramp} \tag{20}$$

Frequency spacing between the points of the FFT:

$$\Delta f = \frac{f_s}{N} = \frac{1}{T_{ramp}} \tag{21}$$

Maximum target speed the radar can measure:

$$v_{\max} = \frac{\lambda}{4\,T_{ramp}} \tag{22}$$

Velocity resolution, where M is the number of ramps used for one measurement:

$$\Delta v = \frac{\lambda}{2\,M\,T_{ramp}} \tag{23}$$

What to do

1. Write a beat frequency function in MATLAB (or set it up in Excel) that takes R and T_{ramp} and returns f_b.
2. Plot f_b against range from 0 to 30 m, with one line for each ramp time. Mark the values at 1 m, 10 m and 30 m.
3. For each ramp time, find the sampling rate needed to see a target at 30 m, and the number of samples per ramp.
4. For each ramp time, calculate the maximum speed $v_{\max}$ in m/s and km/h.
5. Look up how many samples per second the Arduino and the Raspberry Pi can take. Which ramp times could each of them handle? What is the fastest target we could track with each?
6. Decide on a ramp time and a sampling rate for our radar. There is no perfect answer here, so explain your choice.
7. Extra: in MATLAB or Python, make a fake beat signal (a sine wave at the beat frequency for a target at 7 m, with some random noise added), take its FFT, find the peak and convert it back to a range.

What to hand in

Your plots, a table of f_b, f_s, N and $v_{\max}$ for each ramp time, and your recommended ramp time and sampling rate with a short explanation, all on the GitHub.

Task 4: Power Budget and Logic Levels

Background

Every chip on the radar needs electrical power to work. Each one needs a specific **voltage** (think of it as the pressure pushing electricity through) and draws a certain **current** (the amount of electricity flowing). Voltage multiplied by current gives **power**, which is how much energy the chip uses each second.

Our power will come in at 5 V from a USB cable, but our chips need 3.3 V and 1.8 V. Give a chip too much voltage and it is destroyed instantly. So we need **voltage regulators**: small parts that take in a higher voltage and put out a steady lower one. The simple kind (a linear regulator) gets rid of the extra voltage by turning it into heat. If it makes too much heat it overheats, so we have to check.

A wire carrying one particular voltage to several chips is called a **rail**. We will have a 3.3 V rail and a 1.8 V rail. A **power budget** is a table that adds up how much current every chip takes from each rail, so we know how big the regulators need to be.

The second problem is **logic levels**. The Raspberry Pi and Arduino control the radar chips by switching wires between 0 V ("off") and a higher voltage ("on"). The Raspberry Pi uses 3.3 V for "on". Some Arduinos use 5 V. If that "on" voltage is higher than the chip on the other end can survive, it damages the chip. Every datasheet has a table called **Absolute Maximum Ratings** which lists the limits you must never go over.

Where a signal is too high, we can lower it with a **voltage divider**: two resistors in a row, with the output taken from the point between them. For example, two equal resistors cut the voltage in half, so 5 V in gives 2.5 V out.

Values to use

- Input supply: 5 V (from USB)
- The ADF5901 and ADF5904 run from 3.3 V
- The ADF4159 needs 3.3 V and also 1.8 V

Equations

Power used by a chip:

$$P = V\,I \tag{24}$$

Heat produced in a linear voltage regulator:

$$P_{heat} = (V_{in} - V_{out})\,I \tag{25}$$

Temperature rise of a part, where θ_{JA} is the thermal resistance from its datasheet in °C/W:

$$\Delta T = P_{heat}\,\theta_{JA} \tag{26}$$

Voltage divider, where R_1 is the resistor connected to the input and R_2 is the resistor connected to ground:

$$V_{out} = V_{in}\,\frac{R_2}{R_1 + R_2} \tag{27}$$

What to do

1. Find the "Power Supplies" rows in the specifications table of each of the three datasheets. Make a table of every supply pin, its voltage and its current (typical and maximum).
2. Add up the total current on the 3.3 V rail and on the 1.8 V rail, and the total power.
3. Find a voltage regulator for each rail that can supply at least 1.5 times the current you need. Calculate the heat it produces and its temperature rise.
4. Find the "Absolute Maximum Ratings" table in each datasheet and write down the highest voltage allowed on the digital pins of each chip.
5. Compare these with the signal voltages of the Raspberry Pi (3.3 V) and the Arduino (look up your model). Which connections are safe and which are not?
6. For every unsafe connection, design a voltage divider that brings the signal down to a safe level. Use resistors between 1 kΩ and 10 kΩ.
7. Draw a power tree: a block diagram showing the 5 V input, each regulator, and which chips each one feeds, with the current written on every line.

What to hand in

Your current and power table, the power tree drawing, your chosen regulators with links, and your voltage divider designs, all on the GitHub.

Task 5: Test Target and Radar Mount

Background

Once the radar is built we have to test it, and to test it properly we need two things: a target we understand, and a way to hold the radar still.

The target. If we point the radar at a random object and get a weak echo, we cannot tell whether the radar is bad or the object just does not reflect much. So we need a target that reflects a known amount. How much an object reflects is called its **radar cross section** (σ), measured in square metres. A person is about $1\,\mathrm{m}^2$.

The standard test target is a **corner reflector**: three flat metal triangles joined at right angles, like the inside corner of a box. A signal that goes in bounces off the three faces and comes straight back the way it came, no matter what angle it arrived from. This makes it reflect far more than its size suggests, so a reflector you can hold in one hand looks as big as a person to the radar. Its radar cross section depends only on its size and the wavelength, so we can calculate it exactly.

The mount. The radar board needs a holder that keeps it steady and lets us aim it. Three things about the radar affect the design:

- The antennas are flat copper shapes on one side of the board. Nothing metal can be in front of them.
- The receive antennas sit half a wavelength apart, which tells you how small the antenna area is.
- If we put a plastic cover in front of the antennas to protect them, the radar signal has to pass through it. Some of the signal reflects off the front surface of the plastic and some off the back. If the plastic is exactly half a wavelength thick (measured inside the plastic, where the wave is shorter), those two reflections cancel each other out and nearly all the signal gets through. The number that tells you how much shorter the wave is inside a material is its **relative permittivity** (ε_r).

Values to use

- Wavelength λ: get this from Task 1, or use $\lambda = c/f$ with $f = 24\,\mathrm{GHz}$ and $c = 3 \times 10^8\,\mathrm{m/s}$
- Target radar cross sections: $\sigma = 1\,\mathrm{m}^2$ and $\sigma = 10\,\mathrm{m}^2$
- Placeholder board size: a credit card ($85.6\,\mathrm{mm} \times 54\,\mathrm{mm}$)

Equations

Radar cross section of a triangular corner reflector with edge length a (the length of the edges where the faces join):

$$\sigma = \frac{4\pi a^4}{3\lambda^2} \tag{28}$$

Rearranged to give the edge length for the cross section you want:

$$a = \left(\frac{3\lambda^2\sigma}{4\pi}\right)^{1/4} \tag{29}$$

Spacing between neighbouring receive antennas:

$$d = \frac{\lambda}{2} \tag{30}$$

Thickness of a plastic cover that the radar can see through:

$$t = \frac{\lambda}{2\sqrt{\varepsilon_r}} \tag{31}$$

What to do

1. Calculate the edge length a for a $1\,\mathrm{m}^2$ reflector and for a $10\,\mathrm{m}^2$ reflector.
2. Build both from cardboard covered in kitchen foil. Keep the three faces flat and at right angles to each other.
3. Calculate the antenna spacing d, and the total width of four receive antennas in a row.
4. Choose a plastic for the cover, look up its ε_r, and calculate the cover thickness t.
5. In CAD (Onshape or Fusion are free for students), design a holder for the placeholder board. It needs to fit on a camera tripod, tilt up and down, leave the antenna side of the board uncovered or behind your cover, and leave room for cables.
6. Sketch how you would set up a range test: where the radar goes, where the reflector goes, and how you would measure the true distance between them.

What to hand in

The two finished reflectors, your CAD files and a screenshot of the holder, and your calculations, all on the GitHub.